어떤 실패를 두려워하는가: 정적 분석 설계를 가르는 우선순위의 차이

김봄

같은 버그를 탐지하려는 목표에서 출발했음에도 Google, Facebook, Apple이 서로 다른 정적 분석 도구를 설계한 이유는, 각 조직이 가장 두려워한 실패의 종류가 달랐기 때문이다. 이 글에서는 그 차이를 기술적 우열이 아니라 우선순위의 차이로 읽고, 그 우선순위가 인공지능(AI) 보조 도구가 확산되는 오늘날의 개발 환경에서 왜 더 중요한 문제가 되는지를 논한다. 코드 생성 속도가 빨라질수록 어떤 실패를 먼저 막아야 하는지에 대한 판단은 자동화되지 않은 채 더 자주 요구되고, 그 결과 소프트웨어의 올바름을 하나의 기술 기준으로 보장하려는 시도는 점점 어려워진다. 이 글은 그 한계를 극복하는 출발점이 더 강한 도구가 아니라, 우리가 지금 어떤 실패를 가장 감당하기 어렵다고 보는지를 명시적으로 읽어내고 반복적으로 갱신하는 일에 있다고 주장한다.

버그를 탐지한다는 같은 출발점에서 시작했음에도, 세 회사가 서로 다른 도구를 만든 것은 각자 서로 다른 실패를 두려워했기 때문이라고 느껴진다. Google은 경고가 수정으로 이어지지 않는 실패를 가장 나쁜 결과로 보았고, Facebook은 심각한 취약점을 한 번 놓치는 실패를 가장 감당하기 어려운 일로 보았으며, Apple은 개발자가 분석 결과를 끝내 이해하지 못해 따르지 않는 실패를 가장 먼저 막아야 할 문제로 보았다. 그 두려움의 크기가 어느 시점에 경고를 보여줄지, 얼마나 깊이 분석할지, 결과를 어떻게 제시할지를 결정했으리라 생각한다. 나는 이 차이를 기술적 우열이 아니라 우선순위의 차이로 읽었고, 그 우선순위가 어디서 왔는지를 거꾸로 따라가면 각 조직이 실제로 어떤 비용을 감당해왔는지가 보인다고 생각한다. Google의 경우 처리되지 않는 경고가 쌓이는 비용을 이미 세 번의 실패로 겪었고, Facebook은 대규모 서비스에서 놓친 취약점 하나의 파급력을 알고 있었으며, Apple은 이해되지 않은 경고는 결국 무시된다는 것을 경험으로 알고 있었을 것이다.

이러한 우선순위의 차이는, 코드가 많아질수록 어떤 실패를 먼저 막아야 하는지 판단해야 하는 순간도 함께 늘어나지만 그 판단을 자동화할 수 없다는 한계와 직접 맞닿아 있다. 인공지능(AI) 보조 도구가 코드 초안을 만들어서 사용하는 게 익숙해져가는 현대 시대에서, 사람이 처음부터 끝까지 직접 작성하지 않은 코드도 빠르게 검토하고 배포해야 하는 상황이 늘고 있다. 그러나 이 판단을 AI에게 온전히 맡기기 어려운 이유는, 허용 가능한 실패의 범위, 운영 맥락, 수정 비용, 조직이 현재 감당할 수 있는 위험의 크기 같은 정보는 코드 안에 없기 때문이다. 설령 이 정보를 맥락으로 제공한다 해도, 이것들은 고정된 값이 아니라 조직의 상황과 시점에 따라 계속 달라지는 판단이기 때문에 한 번 정해진 기준으로 반복 적용하기 어렵다. AI는 코드에서 신호를 읽을 수 있지만, 그 신호가 지금 이 조직에서 얼마나 나쁜 실패인지는 코드 바깥의 맥락 없이는 판단할 수 없다. 그래서 그 과정에서 판단에 영향을 미치는 요소도 더 복잡해졌고, 무엇을 지금 고쳐야 하고 무엇을 절대 놓치면 안 되며 무엇은 나중으로 미뤄도 되는지 판단해야 하는 부담은 오히려 더 커지고 있다고 생각한다.

그 판단이 하나로 수렴되지 않는다는 것은, 곧 올바름의 기준도 하나로 수렴되지 않는다는 뜻이다. 어떤 환경에서는 배포 후 수정이 어려운 취약점을 사전에 막는 것이 올바름이고, 어떤 환경에서는 경고가 실제 수정으로 이어지는 것이 올바름이며, 어떤 환경에서는 개발자가 분석 결과를 납득하고 따를 수 있는 것이 올바름이다. 하나의 기준을 충족하려 하면 다른 기준과 충돌하는 경우도 많고, 정적 분석은 이 중 어느 하나를 달성하기 위한 조건을 제공할 수 있지만 어떤 조건을 우선할지는 분석 바깥의 판단 주체에게 남는다. 보장해야 할 상태 자체가 맥락마다 다르게 정의되는 한, 하나의 분석 방식이 모든 환경에서 올바름을 보장할 수는 없다고 생각한다.

이 문제를 극복한다는 것은 더 강한 도구를 찾거나 완벽한 기준을 합의하는 일이 아닐 수 있다. 세 사례가 보여주는 것은, 각 조직이 이미 특정 실패를 더 두려워하고 있었고 그 두려움이 도구 설계 전체에 이미 반영되어 있었다는 점이다. 그렇다면 중요한 것은 그 두려움을 명시적으로 읽어내는 것으로, 우리가 지금 어떤 실패를 가장 감당하기 어렵다고 보는지를 인식하고, 그것이 우리의 도구와 프로세스에 실제로 반영되어 있는지를 확인하는 일이라고 생각한다. 그 출발은 거창하지

않아도 된다. 새로운 도구를 도입하기 전에 “우리가 지금 가장 많이 무시하고 있는 경고는 무엇인가”를 먼저 묻거나, 장애나 보안 사고 이후에 어떤 종류의 실패가 실제로 발생했는지를 도구 평가 기준으로 소급해보는 것만으로도, 두려움이 무엇인지를 구체적인 언어로 다루기 시작할 수 있다.

그리고 그 인식은 한 번으로 끝나지 않는다. 시스템이 커지고 배포 방식이 바뀌면, 예전에는 감수할 수 있었던 실패가 더는 감수할 수 없는 실패가 되기도 하고, 반대로 가장 두려웠던 실패가 더 이상 가장 큰 위협이 아니게 되기도 한다. 세 조직이 도구를 계속 바꾸고 측정 기준을 다시 정의해온 것도 그 인식이 반복적으로 갱신되어온 결과라고 볼 수 있다. 그렇다면 새로운 도구를 도입하거나 기존 도구를 평가할 때, 우리가 먼저 물어야 할 질문은 “이 도구가 얼마나 정확한가”가 아니라 “이 도구는 우리가 가장 두려워하는 실패를 줄이는 방향으로 설계되어 있는가”일 것이다. 정적 분석의 성숙이 판단의 종결이 아니라 판단을 더 명시적으로 요구하게 되는 과정이라면, 그 질문을 얼마나 의식적으로 반복할 수 있는지가 결국 소프트웨어의 올바름에 가장 가까이 다가가는 방법이 아닐까 생각한다.